ORCAS — Project Report

Orchestrated Regulatory Complaint Analysis & Routing System

Course	UE24AM342AA5 — ML System Design and AgentOps (PES University, Dept. of CSE, AI & ML)
Project	Mini Project 12 — Securities Investor Grievance Routing Service
Team	Mitesh (agent core & LLM layer) · Tahir (data, retrieval & quality) · Jignas (platform & reliability)
Repository	https://github.com/Mitxh13/orcas

1. Problem and Scope

Retail investors write free-text complaints to SEBI about brokers, mutual funds, registrars, listed companies and depositories. Many of these do not belong to SEBI. A bank fixed-deposit grievance falls under the **RBI Integrated Ombudsman Scheme 2021**, even when the complainant addresses it to SEBI and uses investment vocabulary.

A single retrieval pass on such a complaint returns SEBI's grievance framework, which is the wrong answer. The core problem is **jurisdictional classification under regulatory cross-reference**: a provision in one framework can redirect a complaint to a different regulator, and the redirecting provision can only be found after reading the first evidence and searching again.

Scope of the system

- Accepts a free-text investor complaint via a REST API.
- Decides which regulator (SEBI or RBI) and which regulated entity is responsible.
- Produces a *routing determination* with verified, circular-level citations.
- Escalates to a human reviewer when it cannot be sure; it never guesses.
- Never submits anything to a real regulator portal and never *disposes* of a complaint. Only synthetic complaints are used.

Reference case (must always pass)

> *"I invested in a fixed deposit through my bank's app. The interest credited is lower than what I was promised. I want to complain to SEBI."*

Expected: **refer-to-other-regulator → RBI Integrated Ombudsman Scheme 2021**, with citations that resolve to retrieved text (preamble and clause 9).

2. Agent Design

2.1 Topology

Two independent agents run sequentially inside a single Celery task. The Researcher explores the corpus through a bounded tool-calling loop. The Verifier uses a *read-only* retrieval client (only `get_chunk`; no search) and audits the draft independently. The agents share no mutable state; the hand-off is an explicit contract: *draft + evidence ledger*.

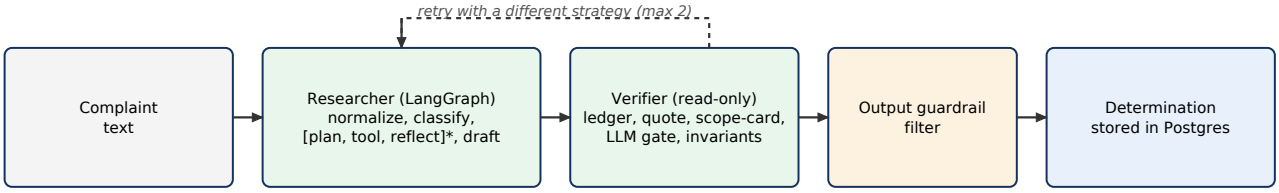


Figure 1. Agent topology and hand-off.

2.2 Tools

Agent	Tool	Purpose
Researcher	`search_corpus(query, framework, entity_type, k)`	Dense + filter retrieval across the SEBI and RBI corpora
Researcher	`get_clause(doc_id, clause_id)`	Exact single-clause lookup for cross-referencing
Researcher	`get_complaint_fields(fields)`	Inspect structured complaint metadata
Researcher	`list_framework_scope(framework)`	Fetch scope cards (finding aids, not citable evidence)
Verifier	`get_chunk(chunk_id, quote_span)`	Resolve a SHA-pinned chunk to audit verbatim substrings

The Verifier cannot call `search_corpus`. This is enforced by wrapping the retrieval client in a `_ReadOnlyRetrievalClient` that exposes only `get_chunk`.

2.3 Verification (5-layer gate)

#	Layer	Rule
1	Ledger containment	Every citation's chunk_id must exist in the evidence ledger retrieved *during this run*. Chunks from earlier runs or invented ids are rejected.
2	Quote span integrity	quote_span must be ≥ 20 characters and appear verbatim (or whitespace-normalised) in the chunk.
3	Scope-card rejection	chunk_type = scope_card chunks are finding aids, not citable authority; rejected unconditionally.
4	LLM grounding gate	The Verifier prompt checks each sentence of the basis against the retrieved passages. Output defaults to passed = False unless the LLM explicitly justifies acceptance.
5	Consistency invariants	refer_to_other_regulator needs a retrieved passage from the target framework; assign_to_entity needs SEBI evidence; timeline.days must appear in the basis-citation text, otherwise the timeline is dropped and noted.

2.4 Loop limits

All bounds are read from environment variables; none are hard-coded. The wall-clock check runs before every LLM call and every tool call. Budget exhaustion produces an *escalated* determination with `reason_code=budget_exhausted`, not a crash.

Parameter	Env var	Default	Rationale
Step limit	`MAX_STEPS`	10	Prevents infinite re-planning
Tool-call budget	`MAX_TOOL_CALLS`	8	Caps retrieval overhead
Wall-clock limit	`RUN_WALL_CLOCK_S`	45 s	Ends well below the Celery soft timeout
Verification retries	`MAX_VERIFY_RETRIES`	2	Allows strategy rotation
Celery soft timeout	`CELERY_SOFT_TIME_LIMIT`	120 s	Graceful degradation before kill
Celery hard timeout	`CELERY_TIME_LIMIT`	180 s	Hard kill

3. Architecture Diagram

Red dashed rectangles mark **deployment boundaries**: the units that are built, scaled and restarted together. Purple marks the observability plane, which is kept separate so monitoring survives an application failure.

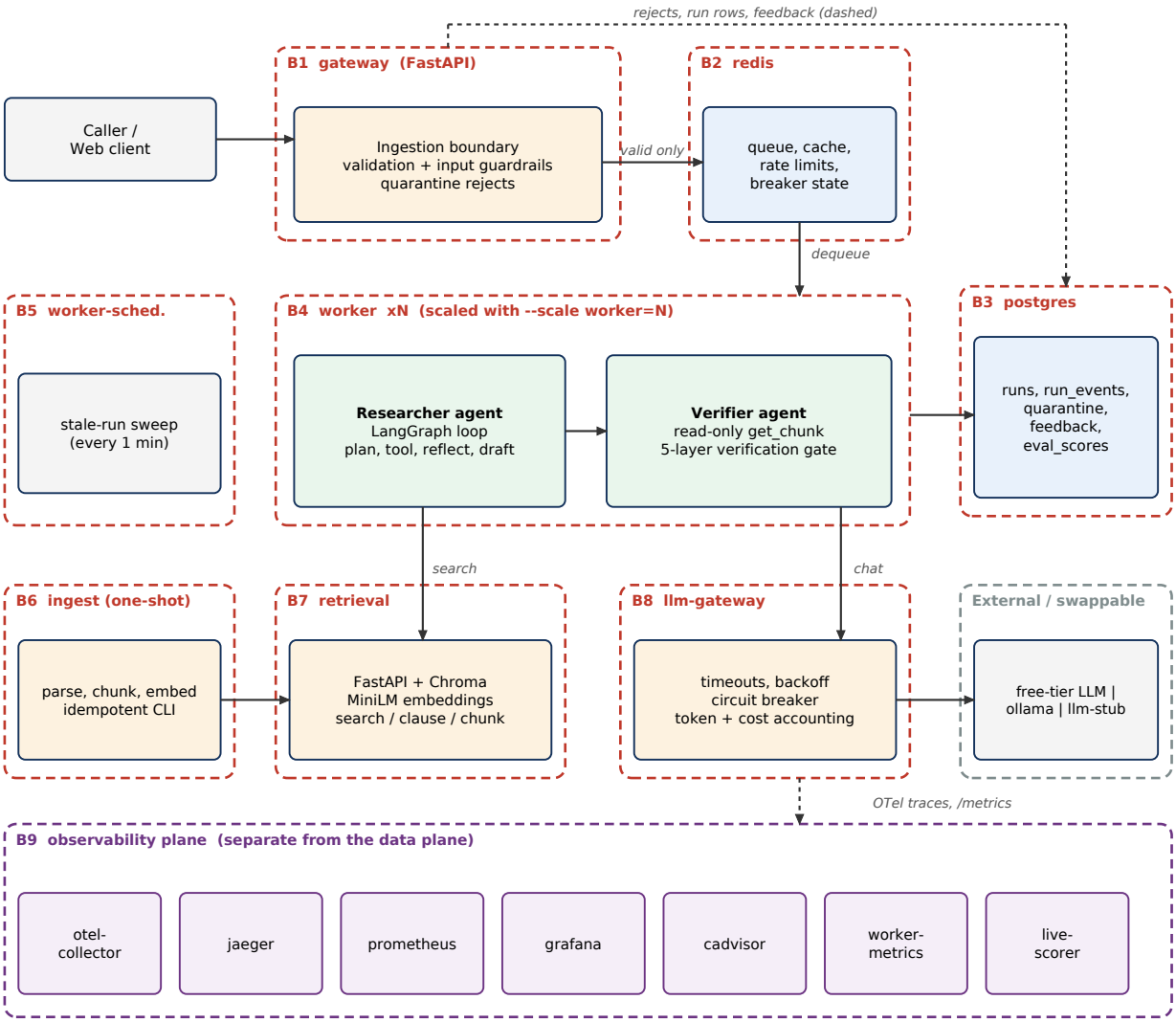


Figure 2. ORCAS architecture with deployment boundaries (B1–B9) marked. Solid arrows are synchronous calls or queue hand-offs (Researcher to Verifier carries the draft and evidence ledger); dashed lines are telemetry or database writes.

Why each boundary exists

Service	Why it is separate
gateway	Latency-sensitive intake; must stay up when agents are slow or crashing.
worker	Long-running and I/O-bound; scaled independently with --scale worker=N.
retrieval	Owns the embedding model and vector index (hundreds of MB); re-indexable without touching agents.
llm-gateway	Only service holding provider credentials; circuit-breaker state is shared across all workers via Redis.
llm-stub	Test isolation: same API as the live upstream, so load tests measure ORCAS, not provider rate limits.
redis	Ephemeral state: queue, cache, rate limits, circuit-breaker state.
postgres	Durable state: runs, events, quarantine, feedback, verification scores.
worker-metrics	Read-only metrics adapter that queries Postgres and Redis for run KPIs without embedding an HTTP server in Celery.
worker-scheduler	Dispatches a stale-run sweep every minute; escalates runs stuck in running.
live-scorer	Scores a sampled fraction of completed runs and writes to eval_scores for Grafana charting.

4. Design Decisions and What Was Rejected

4.1 Two separate agents, not one

Decision: Researcher and Verifier are separate classes with separate LLM clients and separate retrieval permissions, connected by an explicit hand-off contract. **Rejected:** a single agent that researches and self-verifies. Self-verification tends to confirm its own reasoning; an independent verifier with a read-only client cannot introduce evidence the Researcher did not retrieve.

4.2 Clause-aligned chunking, not fixed-window

Decision: regex clause/CHAPTER-aligned segmentation. Each chunk carries a real clause_id, heading_path and inferred entity_types. Long clauses are split into #k sub-chunks. Scope cards (one per framework) are finding aids whose source_clause_chunk_id points to the citable clause.

Rejected: fixed-window chunking. It achieved recall@5 = 1.00 on 6 topical probes against 0.833 for clause-aligned, but the difference is largely an artefact of larger windows trivially containing more phrases. Mean hit score favoured clause-aligned (0.719 vs 0.650); the RBI grounds provision retrieved at 0.809 vs about 0.63. Clause-aligned also yields addressable provisions that verification needs.
Rejected: hybrid retrieval (dense + BM25 + RRF), which showed no measurable recall gain on this corpus; it is available through a mode parameter but off by default.

4.3 LLM failure: fail closed, no heuristic fallback

Decision: any failure to reach the LLM gateway (non-200, timeout, connection refused) raises a typed `LLMUnavailable` exception. The pipeline catches it and produces an *escalated* determination with degraded = true. The Verifier path also fails closed: an unverified draft is never returned as a completed answer. **Rejected:** a local keyword-matching fallback. It made tests pass while masking real connection errors and produced answers that could not be verified. Deleted.

4.4 Prompts as versioned YAML artifacts

Decision: prompts live in `prompts/<name>/<version>.yaml` and are loaded by a PromptLoader with single-pass regex interpolation. Variable names are validated first and the prompt body is never re-parsed after substitution, which prevents injection through complaint text. **Rejected:** f-string templates in code (no versioning, no injection isolation, no audit trail).

4.5 Word-boundary toxicity matching

Decision: \b-anchored regex per toxic word. **Rejected:** substring matching, where “die” wrongly flagged “bodies”, “remedies” and “studied”.

4.6 Cost accounting from environment-driven prices

Decision: `LLM_PRICE_INPUT_PER_M` and `LLM_PRICE_OUTPUT_PER_M` come from the environment and the accounting function never references a model name. Each LLM call event stores prompt tokens, completion tokens and cost; Prometheus exposes `orcas_run_cost_by_step_usd`. **Rejected:** hard-coded per-model price tables, which go stale and do not cover local models.

5. Evaluation Results: Before and After

5.1 Retrieval benchmark: chunking strategy

Six topical probes, framework-filtered.

Strategy	recall@5	recall@10	mean_hit_score
Fixed-window / dense	1.000	1.000	0.650
Clause-aligned / dense (shipped)	**0.833**	**0.833**	**0.719**
Clause-aligned / hybrid	0.833	0.833	n/a

Clause-aligned trades a marginal raw-recall dip for sharper ranking and citable provisions.

5.2 Agent evaluation (stub mode)

The agent (eval/agent_sut.py) was run against 25 scenarios with stub replies, 7 of them held out from tuning. A deterministic embedding judge (an NLI substitute) was used with thresholds: support 0.45, relevance 0.32, inference 0.40.

Metric	All (25)	Held-out (7)
Answered (not escalated)	18	4
Groundedness / faithfulness	1.000 / 1.000	1.000 / 1.000
Context relevance	0.561	0.450
Hallucination rate	0.000	0.000
Route / regulator / entity accuracy	0.720 / 0.720 / 0.680	0.429 / 0.429 / 0.429
Escalation precision / recall	0.714 / 0.500	0.333 / 0.333
Supporting passage retrieved / cited	0.133 / 0.000	0.000 / 0.000
Intake rejection recall / false rejections	1.000 / 0	1.000 / 0

> **Important:** stub replies are canned, so these numbers describe the pipeline structure, not model quality. Groundedness and faithfulness are 1.000 because the stub's single generic sentence aligns with the retrieved passage. No live-model evaluation has been run.

Before / after on the four headline metrics. The keyword baseline (reference only, `make eval-baseline`) against the shipped agent:

Metric	Before (keyword baseline)	After (agent)
Groundedness	0.722	1.000
Context relevance	0.900	0.561
Faithfulness	0.833	1.000
Hallucination rate	0.278	0.000
Supporting passage retrieved	0.333	0.133

> The agent eliminates hallucination, at the cost of lower context relevance and supporting-passage retrieval in stub mode.

5.3 Load test: before and after the stub routing fix

The stub's narrow parsing caused many premature escalations in the first benchmark (10 clients, 1 worker, 1.5 s stub delay). After commit `3a64fc6` hardened classification and routing:

Metric	Before	After
Ten-client p95	40 s	**30 s**
Completed / escalated in DB window	101 / 263	**245 / 136**
LLM calls per terminal run	about 3.2	3.010
Three-worker ten-client p95	20 s	**11 s**
First SLO breach (p95 > 90 s)	40 clients, p95 123 s	40 clients, p95 104 s

6. Performance: p95 Target, Measured Results, Bottleneck and Cost

6.1 SLO target (declared before the first load test)

At ten concurrent clients, p95 submit-to-terminal latency ≤ **90 seconds** and ≥ **99%** of accepted runs reach *completed* or *escalated*. Workload: 1.5-second LLM stub, one worker container with two Celery processes, 100+ complete complaints after warmup.

6.2 Measured results

Workload	Workers	Clients	Obs.	p50	p95	Throughput	Failures
Baseline	1	10	380	22 s	**30 s**	0.417/s	0
Scaled	3	10	1,076	8.2 s	**11 s**	1.186/s	0
Higher load	1	20	144	46 s	52 s	0.414/s	0
Higher load	1	40	164	94 s	**104 s (breach)**	0.418/s	0

SLO result: PASSED at 10 clients (p95 = 30 s, 100% terminal success). The first breach occurs at 40 clients (p95 = 104 s).

Cache effect

Cache mode	p50	p95	Throughput	LLM calls
Repeated input after warmup	0.081 s	0.13 s	9.110/s	0
Same input, bypass enabled	23 s	31 s	0.383/s	255

Warm-cache p95 is about 238× lower than with the cache bypassed; 1,638 cache-hit events were recorded in the repeat window.

6.3 Bottleneck

At 40 clients, sampled worker CPU peaked at 81.03%, gateway 35.23%, Postgres 37.42% and retrieval 3.59% (Docker per-core convention). Queue depth peaked at 38 tasks (mean 30.9). Scaling from 1 to 3 workers cut p95 from 30 s to 11 s and nearly tripled throughput (0.417 → 1.186/s). The evidence points to **limited worker execution slots** (each worker container runs two Celery processes) as the primary bottleneck, not host CPU exhaustion.

6.4 Cost per 100 runs

The stub returns $0 per LLM call. At about 3.0 LLM calls per terminal run, **stub cost = $0 per 100 runs**, a test-environment figure only. The bypass window used 293,199 tokens for 76 runs (about 3,860 tokens per run). Live-model cost per 100 runs is unmeasured until a provider is configured; it must be derived from the persisted `orcas_run_cost_by_step_usd` series.

7. What Did Not Work

1. Silent heuristic fallback. An early LLMClient fell back to keyword matching when the gateway was unreachable. Tests passed while masking real errors, and heuristic answers could not be verified. *Fixed:* LLMUnavailable exception and fail-closed escalation (incident log 2026-11-03).

2. Hard-coded entity branches in the Researcher. A Python check on the word “bank” redirected bank complaints to RBI regardless of agent reasoning, so the reference case passed for the wrong reason. *Fixed:* all hard-coded branches removed; framework swaps are driven by researcher_reflect/v2.yaml reading the retrieved evidence.

3. Substring toxicity matching. “die” flagged legitimate text such as “bodies”, “remedies” and “studied”. *Fixed:* word-boundary regex with explicit false-positive unit tests.

4. Hash replication in small LLMs. Asking models to copy 64-character SHA-256 chunk ids produced truncated hashes and needless escalations. *Fixed:* numbered evidence tags [E1]...[En] with deterministic code-level resolution (researcher_draft/v2.yaml).

5. The stub skewed the first benchmark. Narrow stub classification triggered premature escalations (101 completed vs 263 escalated in the first 10-client run). *Fixed* in commit 3a64fc6; second run 245 completed vs 136 escalated.

6. The first evaluation never scored the agent. make eval ran only the keyword baseline, and groundedness (0.911) was inflated because claims were sentences copied from the retrieved passages. *Fixed:* the harness now scores the agent, with claims taken from the answer’s own text.

7. Automated corpus download was blocked. Firecrawl ran out of credits and the RBI document host's WAF blocks curl. Documents were fetched through a browser, with provenance recorded in corpus/MANIFEST.yaml.

8. Celery task unregistered in the Compose smoke test. A broad ImportError handler in worker tracing suppressed a FastAPI instrumentation failure before the task decorator ran, leaving the run stuck in *queued*. *Fixed:* conditional instrumentation loading; unexpected worker import errors now fail startup (incident log 2026-10-27).

8. References

- Corpus: corpus/MANIFEST.yaml (SHA-256-pinned documents, source URLs, versions)
- SLO: docs/slo.md · Runbook: docs/runbook.md · Incident log: docs/incident_log.md
- Architecture: docs/ARCHITECTURE.md
- Evaluation set: eval/scenarios.yaml (25 scenarios, 7 held out)
- Load-test results: loadtest/results/ci/summary.json
- Detailed sections: docs/work/mitesh/report-sections.md · docs/work/tahir/report-sections.md · docs/work/jignas/report-sections.md